

Maintainable Type Providers for Kotlin

Alexander Kuklev^{1,2} a@kuklev.com

¹Radboud University Nijmegen, Software Science

²JetBrains Research

With type-safe builders, Kotlin provides powerful infrastructure for embedded domain-specific languages enabling accessible declarative specifications for objects and logic specific to various application domains. Kotlin-based DSLs can be used to synthesize data and functions, but not types. Lifting this limitation requires type providers: compile-time functions synthesizing classes, interfaces, and mixins. Type providers enable powerful DSLs including embedded SQL:

```
val users = Db.table("users")
users.select(users.name,
             NewColumn("login") { it.account.name },
             users.age)
    .where { age > 18 and login != null }
```

Unfortunately, type providers have a high potential for abuse, precipitate unwieldy compilation error messages, and greatly complicate debugger and IDE support. Our renewed interest in type providers arises from the observation that the technical problems can be addressed if the generated code is required to be exhaustively annotated and attributed, while the annotation burden reduces the potential for abuse.

1 Constants and compile-time expressions

Type providers are required to be executed at compile-time only, so their arguments have to be compile-time expressions. Kotlin already has the `const` modifier to mark compile-time constants, but they are too limited for our purposes. Currently, constants must have the type `String` or a primitive type. We propose to also support `value` classes wrapping a `String` or primitive value, and `value data` classes, i.e. data classes that contain only strings, primitive values (possibly wrapped into value classes), and `value data` classes. We propose allowing compile-time initializers for constant properties using a syntax similar to setters and getters:

```
const val APP_ENV init() { Comptime.getenv("APP_ENV") ?: "DEV" }
```

The `init` blocks may only invoke methods of the respective APIs of the compiler and compiler plugins while being entirely self-contained otherwise. We also propose to allow marking function arguments (and receiver type in case of extension functions) `const`. Such arguments also must be instantiated by compile-time constants, i.g. expressions that (after inlining all invocations of inline functions) only contain literals, primitive operations and other identifiers marked `const`.

2 Type providers

We propose the following syntax for declaring type providers:

```
init data class fun ExprCtx.JsonClass(JsonSchema schema) : this.NewClass<Any> {...}
init class fun ExprCtx.KtDatabase(schemaSource : String) : this.NewClass<DbBase> {...}
```

Now given compile-time expressions `c` of suitable types, one can use `JsonClass<[c]>` and `KtDatabase<[c]>` as types in strictly covariant positions (e.g. as return types and out type parameters), and to inherit objects, classes and, if applicable, also interfaces from. In the case of generated final classes (like `KtDatabase`) above, only objects can be inherited:

```
fun const JsonSchema.parseJson(json : String) : JsonClass<[this]> {...}

object Db : KtDatabase<["kqdb:ourApp.prototypeDb"]> {
    this(endpoint = Config.dbEndpoint)
}
```

Whenever the compiler encounters such an expression, it executes the respective function in compile-time passing the context at the expression call site as `this : ExprCtx`. The respective functions have to generate a declaration of the class or interface with specified modifiers, attributing each of its non-static subexpression to the specific place in its arguments it was generated from, e.g. `JsonSchema("{properties: {name: {type: ['string']}}}")` should generate `data class That(val name : String)`, while `KtDatabase(s)` would simply yield

```
class That(args) : DbBase(dbSchema, args) {
    companion object {
        const val dbSchema init() { Comptime["KtDatabasePluign"].retrieveSchema(s) }
    }
}
```

To properly understand type providers, it is crucial to realize that they produce anonymous types exactly like object expressions `object {...}` that can never be used in invariant or contravariant positions. Otherwise, we would allow perfectly innocent statements producing perfectly valid type mismatch errors:

```
val x : SomeTypeProvider<[c]> = SomeTypeProvider<[c]>(args)
```

The anonymous type produced by `SomeTypeProvider` on the left is nominally different from the type it produces on the right. Our restrictions enforce correct usage, e.g.

```
class C(args) : SomeTypeProvider<[c]>(args) {}
val x : C = C(args)
// or
object X : SomeTypeProvider<[c]>(args) {}
// or
val x = object : SomeTypeProvider<[c]>(args) {}
```

In the first two cases, we use the type provider for declaring named (non-anonymous) types, while the third one sets the right expectations: object expressions are already known to produce anonymous types and can be perfectly dealt with using the expression `SomeTypeProvider<[c]>` in covariant-only positions:

```
fun f() : SomeTypeProvider<[c]> {
    return object : SomeTypeProvider<[c]>(args)
}
```

Let us now demonstrate what the signature of embedded SQL-methods looks like:

```
abstract class DbBase(const val dbSchema : , args) {
    public fun table(const name : String) : Table<[name]>

    internal init class fun ExprCtx.Table(name : String)
        : this.NewClass<this@DbBase.Table> {...}

    abstract inner class Table : View
    abstract inner class View {
        public fun View.select(vararg const cols : this@DbBase.ColSpec) : View<[cols]>

        internal init class fun ExprCtx.View(cols : this.ColSpec)
            : this.NewClass<this@DbBase.View> {...}
        ...
    }
    ...
}
```

For best results, we need abstract type members (abstract inner class) as in Scala 3.

3 Compile-time dependent types

Many modern functional programming languages support dependent types, i.e. types that can have non-typal parameters. Sometimes these can be only used as non-introspectable ('erasable') indexes, which is known as lightweight dependent types, but full-fledged dependent types are essentially functions that use their parameters to compute a type, which might remind one of type providers. Dependent types are required to be genuine functions yielding the same result for the same parameters, so they cannot be used to generate new classes; they only compute valid type expressions instead. While quite distinct from type providers in both purpose and nature, compile-time dependent types functions can be implemented using the same machinery:

```
val x : ByName("Int") = 5

init typealias fun ExprCtx.foo(name : String)
: this.InvariantTypeExpr = when(name) {
  "Int" -> Int::class
  "String" -> String::class
  else -> Any::class
}
```

Providing type-safe signatures for printf-like functions is the most relevant use cases for compile-time dependent types, and these cases deserve a special syntax:

```
init vararg fun PlaceholderTypes(fmt : String) : Sequence<ArgDecl> = {...}

fun const String.format(vararg args : *PlaceholderTypes<[this]>) {...}
// ("It costs %4.2f, \s").format(price, username)
```

Here, the function `PlaceholderTypes` would determine the required types for the placeholders in the format string `this`, and produce a list of corresponding argument declarations. Note that the type of `PlaceholderTypes`. It's not a list of types, but a sequence (allowing `vararg` inside of `vararg`) of argument declarations allowing named arguments:

```
("It costs %{price}4.2f, \{name}s").format(price = 5.99, name = username)
```

We want to allow position-only and name-only arguments as in Python, thus

```
data class ExprCtx.UnnamedArgument(val type : this.InvariantTypeExpr,
                                   val optional : Boolean = false) : this.ArgDecl

data class ExprCtx.NamedArgument(val name : SimpleIdentifier,
                                 val type : this.InvariantTypeExpr,
                                 val optional : Boolean = false
                                 val nameOnly : Boolean = false) : this.ArgDecl
```

To allow signatures like `fun f(cv : Canvas, p : cv.Point)`, the next arguments' types may depend on previous arguments:

```
fun interface ArgDecl { fun ExprCtx.nextArg() : this.ArgDecl }
```

4 Conclusion and outlook

Here be dragons